1. Create a Simple Login Form for a Mock Zomato Admin Panel

Python Code:

```
admin_username = "admin"
admin_password = "zomato123"

username = input("Enter username: ")
password = input("Enter password: ")

if username == admin_username and password ==
admin_password:
    print("Access Granted")
else:
    print("Access Denied")
```

Explanation:

This program asks the user to enter a username and password. It compares the entered details with the hardcoded admin credentials. If both match, it displays **Access Granted**; otherwise, it displays **Access Denied**.

Sample Output:

```
Enter username: admin
Enter password: zomato123
Access Granted
```

Note: This is a practice example. Real applications should not store passwords directly in source code.

2. Create a Python Function for Access Control

Python Code:

```
def check_access(user_role, resource):
    permissions = {
        "customer": ["menu_view", "order_history"],
        "restaurant_owner": ["menu_view",
"menu_edit", "order_history"],
        "admin": ["menu_view", "menu_edit",
"order_history", "admin_dashboard"]
    }

    if user_role in permissions:
        return resource in permissions[user_role]

    return False


# Test cases
print(check_access("customer", "order_history"))
print(check_access("customer", "admin_dashboard"))
print(check_access("restaurant_owner", "menu_edit"))
print(check_access("restaurant_owner",
"admin_dashboard"))
print(check_access("admin", "admin_dashboard"))
```

Output:

```
True
False
True
False
True
```

Explanation:

This function uses a dictionary to store the resources allowed for each user role. It returns `True` if the role has permission to access the requested resource; otherwise, it returns `False`. Customers can view menus and their order history, restaurant owners can edit menus, and admins can access the admin dashboard.

3. Simulate Two-Factor Authentication (2FA)

Python Code:

```
import random

correct_password = "paytm123"

password = input("Enter your password: ")

if password == correct_password:
    otp = random.randint(100000, 999999)

    print("Your OTP is:", otp)

    entered_otp = input("Enter the OTP: ")

    if entered_otp == str(otp):
        print("Login Successful")
    else:
        print("Invalid OTP")
else:
    print("Incorrect Password")
```

Explanation:

This program simulates two-factor authentication. First, the user must enter the correct password. The

program then generates a random six-digit OTP and displays it in the console for testing. The user must enter the same OTP to complete login successfully. If the password or OTP is incorrect, access is denied.

Sample Output:

```
Enter your password: paytm123
Your OTP is: 482615
Enter the OTP: 482615
Login Successful
```

Note: This is a learning simulation. Real applications should send OTPs through a secure, verified channel instead of displaying them in the console.

4. Filter Premium Users Who Logged In Within the Last 7 Days

JavaScript Code:

```
const users = [
{ username: "Rahul", isPremium: true, lastLogin:
"2024-06-09" },
{ username: "Priya", isPremium: false, lastLogin:
"2024-06-08" },
{ username: "Amit", isPremium: true, lastLogin:
"2024-06-01" },
{ username: "Neha", isPremium: true, lastLogin:
"2024-06-05" },
{ username: "Karan", isPremium: true, lastLogin:
"2024-06-10" }
];

function filterPremiumUsers(users) {
const today = new Date("2024-06-10");
const sevenDaysAgo = new Date("2024-06-03");

return users.filter(user => {
const loginDate = new Date(user.lastLogin);

return user.isPremium &&
loginDate >= sevenDaysAgo &&
loginDate <= today;
});
}

console.log(filterPremiumUsers(users));
```

Output:

```
[
{ username: "Rahul", isPremium: true, lastLogin:
"2024-06-09" },
{ username: "Neha", isPremium: true, lastLogin:
"2024-06-05" },
{ username: "Karan", isPremium: true, lastLogin:
"2024-06-10" }
]
```

Explanation:

This function filters the user array and returns only users who have a premium account and logged in within the last seven days. The date range is from June 3, 2024, to June 10, 2024, inclusive. Users who are not premium or whose last login falls outside this date range are excluded.

5. Common Access Control Mistakes and Their Prevention

Access control ensures that users can access only the data and features they are authorized to use. The following are five common mistakes developers make in real-world applications like Instagram and Flipkart.

1. Broken Access Control

Mistake: Users can access another user's private data without permission.

Prevention: Verify the user's identity and permissions on the server before allowing access to private data.

2. Missing Role-Based Permissions

Mistake: Normal users can access admin features or perform restricted actions.

Prevention: Implement Role-Based Access Control (RBAC) to assign permissions according to user roles.

3. Insecure Direct Object References (IDOR)

Mistake: A user changes an order ID or profile ID in a URL to access another user's information.

Prevention: Check whether the requested record belongs to the logged-in user or whether the user has explicit permission to access it.

4. Trusting Client-Side Validation

Mistake: Developers rely only on frontend checks to restrict access to sensitive features.

Prevention: Enforce all important authorization checks on the server because client-side checks can be bypassed.

5. Weak Session Management

Mistake: Poorly protected login sessions may allow unauthorized users to access an account.

Prevention: Use secure session cookies, HTTPS, session expiration, and proper logout functionality.

Conclusion

Proper authentication and access control help protect user accounts, private information, and administrative features. Developers should follow the principle of least privilege, validate permissions on the server, and use secure session management to reduce unauthorized access.